Module 3 – Performance Optimization

1. Memoization

What is Memoization?

Memoization is a technique used to improve performance by caching the results of expensive computations and reusing them when the same inputs occur.

Why It Matters

• Prevents unnecessary recalculations
• Improves application speed
• Reduces rendering cost

---

2. React.memo

What is React.memo?

React.memo is a higher-order component that prevents unnecessary re-rendering of components.

How It Works

• Compares previous props with new props
• Re-renders only if props change

Example:

```
import React from "react";

const Child = React.memo(({ name }) => {
  console.log("Rendered");
  return <h1>{name}</h1>;
});
```

---

3. useMemo Hook

What is useMemo?

useMemo is used to memoize values and avoid expensive calculations on every render.

Example:

```
import { useMemo } from "react";

const result = useMemo(() => {
```

```
  return expensiveFunction(data);
}, [data]);
```

Key Points

• Runs only when dependencies change
• Improves performance in heavy computations

---

4. useCallback Hook

What is useCallback?

useCallback returns a memoized version of a function.

Why Use It

• Prevents unnecessary function recreation
• Useful when passing functions as props

Example:

```
import { useCallback } from "react";

const handleClick = useCallback(() => {
  console.log("Clicked");
}, []);
```

---

5. Lazy Loading

What is Lazy Loading?

Lazy loading allows components to load only when they are needed.

Benefits

• Faster initial load time
• Better performance
• Reduced bundle size

Example:

```
import React, { lazy, Suspense } from "react";

const LazyComponent = lazy(() => import("./MyComponent"));

function App() {
  return (
    <Suspense fallback={<div>Loading...</div>}>
```

```
      <LazyComponent />
    </Suspense>
  );
}
```

---

6. Key Concepts to Remember

• Memoization improves performance by caching results
• React.memo prevents unnecessary re-renders
• useMemo optimizes expensive calculations
• useCallback optimizes functions
• Lazy loading reduces initial load time

---

7. Summary

Performance optimization in React is essential for building fast and scalable applications. By using techniques like memoization, React.memo, hooks like useMemo and useCallback, and lazy loading, developers can significantly improve application efficiency and user experience.